

Programming Principles I

Week 01.



COMT

SOFTWARE CLUSTER

// COMT • **COMPUTATIONAL THINKING** (CIT1C18)



Overview

- Learn how to write a simple Java application;
- Learn how to compile and run a Java application
- Learn the concept of Variables and Data Types.



Objectives

- Understand the structure of a Java application;
- Demonstrate how to write a simple Java application;
- Demonstrate how to compile and run a Java application
- Demonstrate how to create variables with appropriate data type.



JAVA

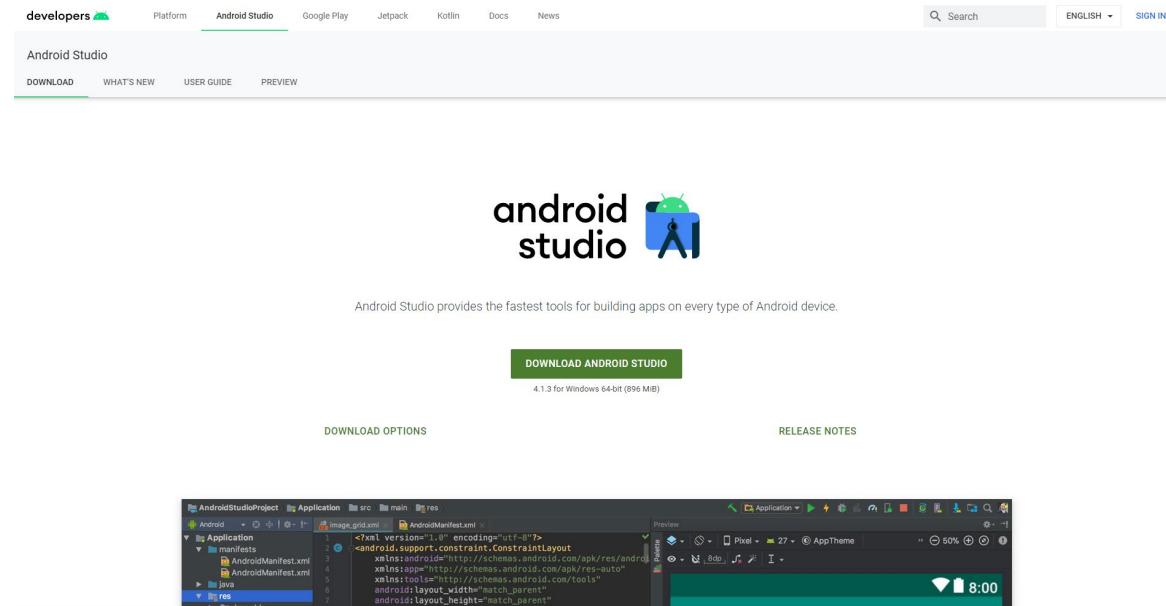


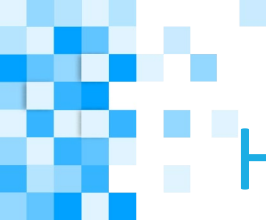
What is Java?

- Java is a general-purpose computer-programming language that is concurrent, class-based, and object-oriented.
- It is intended to let application developers "write once, run anywhere" (WORA), meaning that compiled Java program can run on all platforms (such as Windows, Mac, Linux) that support Java.

Setting up the IDE to learn Java

- We will be using the Android Studio to learn Java
- Refer to the detailed video on creating your first Java program in android Studio





History of Java

- 1991 - James Gosling, Mike Sheridan, and Patrick Naughton initiated the Java language project in June.
- 1995 - Sun Microsystems released the first public implementation as Java 1.0.
- JDK 1.1 (February 19, 1997)
- J2SE 1.2 (December 8, 1998)
- J2SE 1.3 (May 8, 2000)
- J2SE 1.4 (February 6, 2002)
- J2SE 5.0 (September 30, 2004)
- Java SE 6 (December 11, 2006)
- 2010 – Oracle acquired Sun Microsystems
- Java SE 7 (July 28, 2011)
- Java SE 8 (March 18, 2014)
- Java SE 9 (September 21, 2017)
- Java SE 10 (March 20, 2018)



JAVA STRUCTURE

Structure of Java Application

- Below is a simple Java application called **MyJavaApp**.

The **class** keyword tells the system that this is a blueprint of a Java application/object. We will talk about **object** later in this course.

You must give every Java application/object a name. The application/object name that you define must be written after the **class** keyword. It is also called the **class name**.

This is the **main** method that makes a Java application complete. This is the entry-point where all instructions start to get executed.

The open and close braces define the start and end of a program. All program codes must be written within the braces.

```
1 public class MyJavaApp
2 {
3     public static void main(String[] args)
4     {
5         String name = "John";
6
7         System.out.println("Hello World!");
8         System.out.println("My name is " + name);
9     }
10 }
```

Line 5 here is the first instruction that will be executed when the application runs. Lines 7 and 8 will be executed thereafter. Note that every line of instruction must end with a semi colon: ;

Structure of Java Application

A Java application can contain one or more methods. The open and close braces in each method define the start and end of a method.

```
public class MyJavaApp
```

```
{
```

```
    public void methodOne()
```

```
    {
```

```
    }
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String name = "John";
```

```
        System.out.println("Hello World!");
```

```
        System.out.println("My name is: " + name);
```

```
    }
```

```
}
```

Methods are created by you to perform a certain set of instructions.

This example here is a method called **methodOne** that contains no instruction. We will explain more about methods later in this course.

This is called the **main** method and is required in every Java application. A Java application starts executing instructions from here line-by-line.



Java Application File

- A Java application is written in a text file as shown in the previous example.
- A Java application file must be saved with **.java** extension. The filename must be the same as the application/class name, and it must start with upper case letter following industry standard.
- For instance, the previous example Java application is named **MyJavaApp**. Hence, the file must be saved as **MyJavaApp.java**.
- If you use a different name for the application such as **MyApp**, then the file must be saved as **MyApp.java**.
- Proceed to next slide to learn about the rules of naming an application/object aka class name.

Rules of Application/Object Names aka Class Names

- The following lists the correct and wrong ways of naming an application/object aka class name.

X

My Java App

2MyJavaApp

#MyJavaApp

myJavaApp

myjavaapp

12

✓

MyJavaApp

MyJavaApp2

_MyJavaApp



VARIABLES

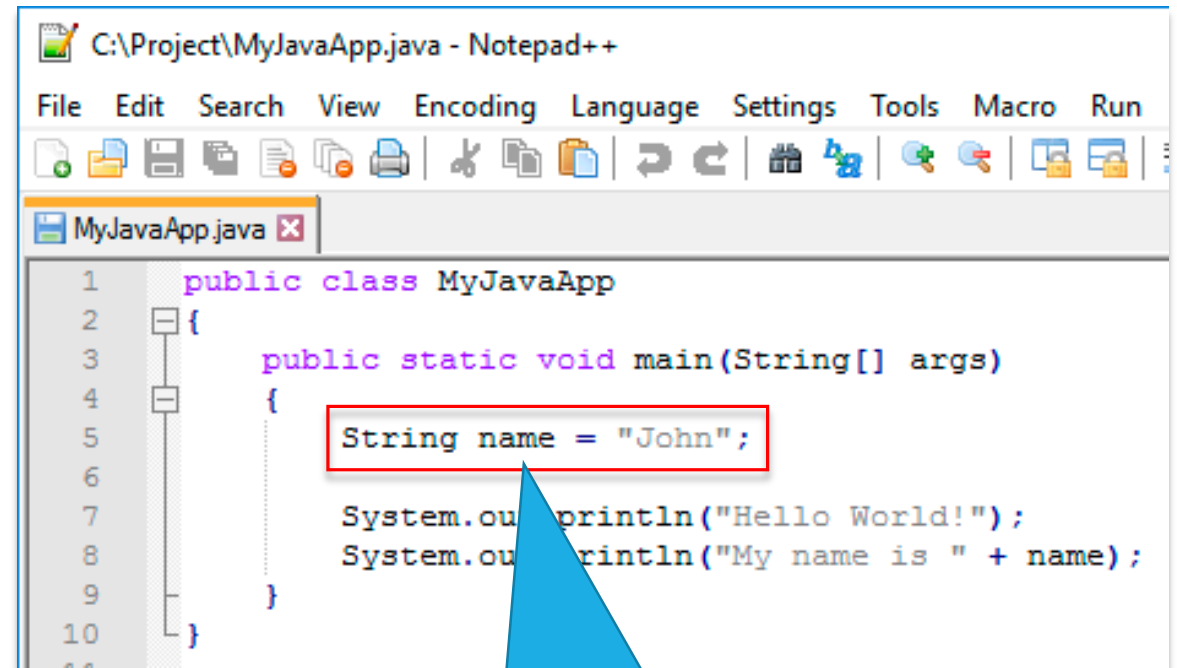


What are Variables?

- Variables are the fundamental building blocks of an application.
- Without variables, very limited functionalities can be created in an application.
- As a human, we can remember names, numbers, objects because they are stored in our memory. We can even perform simple calculations using the data stored in our memory!
- Likewise, we also need a way to store names, numbers, objects in a computer memory.
- Variables are the memory location where data is stored in computers.

What are Variables?

- After we store a data in a variable, how does the computer know where to retrieve it?
- To solve this problem, we will need to give a name to every variable that we create.
- For instance, in the **MyJavaApp** example, we want to store a name called “John” and so we created a variable called **name**. You can create a variable called **a** or **b** or **xy** but these names will not be meaningful to identify what data is being stored.
- The **=** sign means that the word “John” is to be stored in the variable called **name**.
- Notice that when we store a word, we need to surround the word using double quotes: **“John”**.
- This is to help the computer to differentiate between a variable name and the data that is stored.
- Hence, **variable names will never be surrounded by any quotes**.

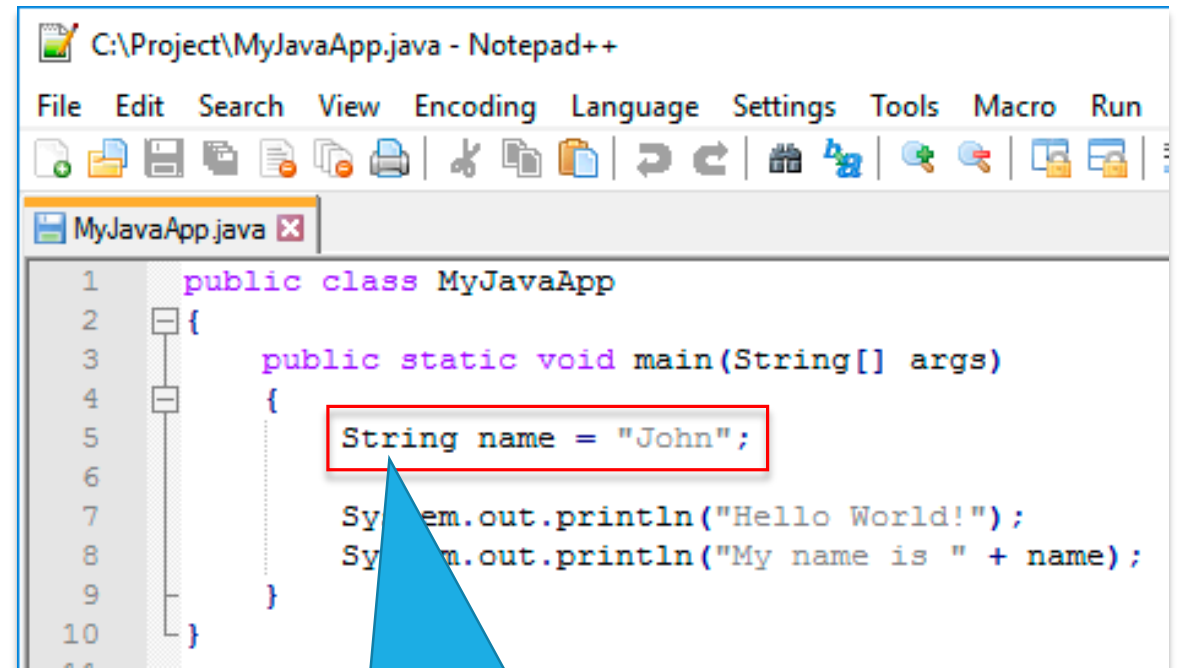


```
C:\Project\MyJavaApp.java - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run
MyJavaApp.java x
1 public class MyJavaApp
2 {
3     public static void main(String[] args)
4     {
5         String name = "John";
6
7         System.out.println("Hello World!");
8         System.out.println("My name is " + name);
9     }
10 }
```

This is the variable called **name**.

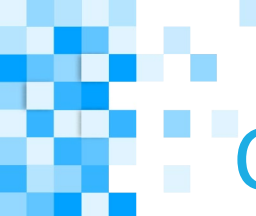
What are Variables?

- Now, other than words, we can also store numbers in a computer memory. How then can the computer differentiate between numbers and words?
- To solve this problem, we need to let the computer know what type of data is being stored.
- In the **MyJavaApp** example, we do this by placing a provided **data type** called **String** before the variable **name** as shown in the code.



```
C:\Project\MyJavaApp.java - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run
MyJavaApp.java x
1 public class MyJavaApp
2 {
3     public static void main(String[] args)
4     {
5         String name = "John";
6
7         System.out.println("Hello World!");
8         System.out.println("My name is " + name);
9     }
10 }
```

This is the data type called **String**.



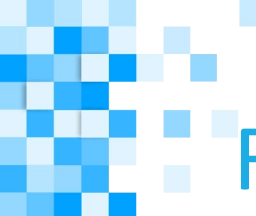
Creating Variables

- Hence, to create any variable, use the following format:

`<data type> <variable name> = <data value>`

`String name = "John"`

- Proceed to the next slide to learn about the rules of naming variables.



Rules of Variable Names

- The following lists the correct and wrong ways of naming a variable.

X

```
Student Name  
2StudentName  
#StudentName  
StudentName  
Studentname  
12
```

✓

```
studentName  
studentName2  
_studentName
```



Data Types

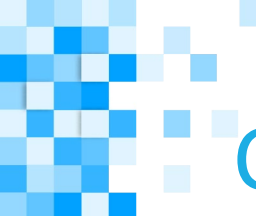
- The first **data type** that you have learned is `String`.
- The following are the kind of data that you can store with `String`:
 - An **alphabet** – e.g. “B”
 - One or more **words** – e.g. “love” or “Good day!”
 - One or more **sentences** – e.g. “How old are you?” or “The day is long. Time for a break!”
- Remember that the data for `String` **must be surrounded with double quotes!**
- There are many **data types** available for use in Java.
- The next few slides will introduce you to some of the other commonly used **data types**.

Commonly Used Data Types

Data Types	Description	When to Use	Example
int	• <code>int</code> stands for Integer • <code>int</code> data type is a 32-bit signed two's complement integer. • Minimum value is - 2,147,483,648 (-2^{31}) • Maximum value is 2,147,483,647(inclusive) ($2^{31} - 1$) • The default value is 0	Integer is generally used as the default data type for integral values unless there is a concern about memory.	<pre>int a = 100000; int b = -200000;</pre>
long	• <code>long</code> data type is a 64-bit signed two's complement integer • Minimum value is -9,223,372,036,854,775,808(-2^{63}) • Maximum value is 9,223,372,036,854,775,807 (inclusive)($2^{63} - 1$) • Default value is 0L	This type is used when a wider range than <code>int</code> is needed. One example would be the distance between planets across galaxies.	<pre>long a = 100000L; long b = -200000L;</pre>

Commonly Used Data Types

Data Types	Description	When to Use	Example
float	<code>float</code> data type is a single-precision 32-bit IEEE 754 floating point - Default value is 0.0f	<code>float</code> is mainly used to save memory in large arrays of floating point numbers. <code>float</code> data type is never used for precise values such as currency.	<pre>float f1 = 234.5f;</pre>
double	<code>double</code> data type is a double-precision 64-bit IEEE 754 floating point - Default value is 0.0d	- This data type is generally used as the default data type for decimal values, generally the default choice. <code>double</code> data type should never be used for precise values such as currency. However, for normal calculation of prices, measurements, etc.. <code>double</code> can still be used.	<pre>double price = 123.48;</pre>



Commonly Used Data Types

Data Types	Description	When to Use	Example
boolean	• <code>boolean</code> data type represents one bit of information • There are only two possible values: <code>true</code> and <code>false</code> • Default value is <code>false</code>	This data type is used for simple flags that track true/false conditions.	<pre>boolean status = true;</pre> *Notice that there are no quotes surrounding the value <code>true</code> or <code>false</code>.
char	• <code>char</code> data type is a single 16-bit Unicode character • Minimum value is <code>'\u0000'</code> (or 0) • Maximum value is <code>'\uffff'</code> (or 65,535 inclusive)	<code>char</code> data type is used to store any character.	<pre>char letterA = 'A';</pre> *Notice that the character is surrounded by single quote: <code>'A'</code>

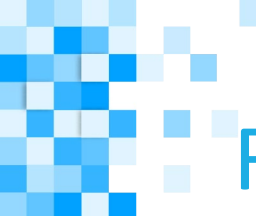


Fun Quiz

```
String price1 = "12.90";
```

```
double price2 = 12.90;
```

- In the above examples, does `price1 = price2`? Why?



Fun Quiz

```
1 public class MyJavaApp
2 {
3     public static void main(String[] args)
4     {
5         String name = "John";
6
7         System.out.println("Hello World!");
8         System.out.println("My name is " + name);
9     }
10 }
```

- In the **MyJavaApp** example as shown above, what do you think is the function of the plus (+) sign in the highlighted line?

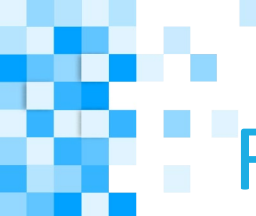


Fun Quiz

```
String price1 = "12.90";
```

```
double price2 = 12.90;
```

- In the above examples, what is the result of `price1 + price2`?
 - 12.9
 - 25.8
 - 12.9012.9
 - 26



Fun Quiz

```
int age = 13;
```

```
int years = 5;
```

- In the above examples, what is the result of `age + years`?
 - 18
 - 135
 - 13
 - 5



Fun Quiz

```
String grade1 = "B";
```

```
char grade2 = 'B';
```

- In the above examples, does `grade1 = grade2`? Why?

Challenge

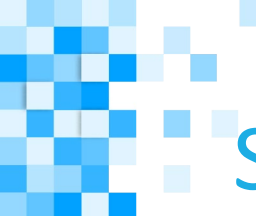
- In `MyJavaApp.java` file, add the following variables using appropriate name and data type to store the data in bold, and then output the sentence to the console:
 1. The length of this table is **1.6m** (See below for solution of this as an example)
 2. The name of this flower is **Angelica**
 3. The price of an iPhone is **\$1199.99**
 4. The age of a student is **17** years old
 5. The grade of Chemistry is **'B'**
 6. It is **true** that the rain has stopped

```
1 public class MyJavaApp
2 {
3     public static void main(String[] args)
4     {
5         String name = "John";
6         double lengthOfTable = 1.6;
7
8         System.out.println("Hello World!");
9         System.out.println("My name is " + name);
10        System.out.println("The length of this table is " + lengthOfTable + "m");
11    }
12 }
13 }
```



Summary

- In this lesson, you have learned the concepts of **Variables** and **Data Types**.
- In the next lesson, you will learn how to manipulate variables using **operators**; and how to determine **conditions** by writing **conditional statements**.



Supplementary Materials

Topic	Source
Java	https://www.javaworld.com/article/2076075/learn-java/core-java-learn-java-from-the-ground-up.html https://www.javaworld.com/article/2925720/learn-java/elementary-java-language-features.html
Java Classes and Objects	https://www.javaworld.com/article/2979739/learn-java/java-101-classes-and-objects-in-java.html
Java Variables	https://www.tutorialspoint.com/java/java_variable_types.htm

THE END



COMT